

Explication de l'application Flutter — TFC Système Multicanal

Guide pas-à-pas pour quelqu'un qui ne connaît pas Dart/Flutter

Document préparé pour Samuel Mumbere Mathe — ULPGL

Juillet 2026

Table des matières

Introduction	1
Partie 1 — Les bases de Dart qu'il faut connaître	2
Une variable typée, comme en C++	2
final et const	2
Les classes	2
async, await, Future	2
Le point d'interrogation ? et le point d'exclamation ! (null safety)	3
L'interpolation de texte \$variable	3
Les widgets : StatelessWidget et StatefulWidget	3
setState(() { ... })	3
build(BuildContext context)	4
Partie 2 — models.dart : la traduction du JSON en objets Dart	4
Partie 3 — api_client.dart : parler au firmware	4
Partie 4 — Les écrans	5
home_screen.dart — l'écran d'accueil	5
tenant_screen.dart — l'écran locataire	6
admin_screen.dart — l'écran bailleur/admin	7
main.dart — le point de départ	7
Partie 5 — Comment tout s'articule	8
Glossaire rapide	8

Introduction

Ce document explique le code de l'application mobile (application/lib/), fichier par fichier, sans rien supposer connu en Dart (le langage de Flutter). Comme pour le firmware, l'objectif n'est pas de te transformer en développeur Flutter, mais que tu puisses expliquer devant le jury ce que fait chaque écran et comment l'application parle au firmware.

L'application a 6 fichiers, organisés ainsi :

lib/

└ main.dart	point d'entree de l'application
└ models.dart	les "fiches" de donnees (miroir du JSON du firmware)
└ api_client.dart	les appels reseau vers l'ESP32
└ screens/	
└ home_screen.dart	ecran d'accueil (choix locataire/admin)
└ tenant_screen.dart	ecran locataire (lecture seule)
└ admin_screen.dart	ecran bailleur/admin (vue + recharge)

Partie 1 — Les bases de Dart qu'il faut connaître

Une variable typée, comme en C++

```
int canal = 0;
double courant = 3.2;
String role = "locataire";
bool relaisFerme = true;
```

Même principe qu'en C++ (vu dans le document firmware) : le type est écrit avant le nom. double = nombre décimal (équivalent du float en C++, juste plus précis).

final et const

final veut dire *"cette variable ne changera plus une fois remplie"* — similaire à const en C++, mais peut être calculée au moment de l'exécution (const en Dart doit être connu à l'avance, avant même de lancer le programme).

```
final String baseUrl; // rempli une fois, ne change plus
```

Les classes

Une classe, c'est la version Dart d'une struct C++, mais en plus complet : elle peut aussi contenir des fonctions (appelées "méthodes").

```
class CanalEtat {
    final int canal;
    final double courantA;

    CanalEtat({required this.canal, required this.courantA});
}
```

CanalEtat({required this.canal, ...}) est le **constructeur** : la recette pour fabriquer un objet CanalEtat. required veut dire *"ce champ est obligatoire à la création"*.

async, await, Future

Un appel réseau (demander une page à l'ESP32) prend du temps — l'écran ne doit pas se figer en attendant la réponse. Dart gère ça avec :

- Future<T> : *"une valeur de type T qui arrivera plus tard"* (le temps que la requête réseau se termine)
- async : marque une fonction comme *"contient des opérations qui prennent du temps"*
- await : *"attends que ce Future soit prêt avant de continuer, mais sans bloquer le reste de l'application pendant ce temps"*

```
Future<EtatSysteme> fetchEtat() async {
  final reponse = await http.get(uri); // attend la reponse du serveur
  return EtatSysteme.fromJson(...);    // s'exécute après la reponse
}
```

C'est l'équivalent direct de `fetch(...).then(...)` ou `async/await` en JavaScript, si tu as déjà vu ça dans `app.js` de l'interface web.

Le point d'interrogation ? et le point d'exclamation ! (null safety)

Dart oblige à dire explicitement si une variable **peut être vide** (null, "rien") ou non :

```
CanalEtat? _canal; // peut valoir null (pas encore de données reçues)
String? _erreur;  // peut valoir null (pas d'erreur actuellement)
```

Le `?` après le type veut dire "cette variable peut être null". Le `!` utilisé plus loin (ex: `_erreur!`) veut dire "je te garantis que ce n'est pas null ici, fais-moi confiance". C'est une protection intégrée au langage contre un type d'erreur très fréquent dans d'autres langages ("null pointer exception").

L'interpolation de texte \$variable

```
Text("Canal ${c.canal} (${c.role})")
```

`$variable` ou `${expression}` à l'intérieur d'un texte insère directement la valeur — équivalent des f-strings en Python (`f"Canal {c.canal}"`) ou des template strings en JavaScript (``Canal ${c.canal}``).

Les widgets : StatelessWidget et StatefulWidget

En Flutter, **tout ce qui s'affiche à l'écran est un "widget"** — un bouton, un texte, une mise en page, un écran entier. Il y a deux familles :

- **StatelessWidget** : un widget qui ne change jamais après avoir été dessiné (ex : le petit widget `_Ligne` dans `tenant_screen.dart`, qui affiche juste un label et une valeur fixes).
- **StatefulWidget** : un widget qui peut changer en réaction à un événement (une réponse réseau qui arrive, un bouton cliqué...). Il est toujours accompagné d'une classe "State" associée (ex : `_HomeScreenState`), qui contient les données qui peuvent changer.

```
class TenantScreen extends StatelessWidget { ... } // la "coquille"
class _TenantScreenState extends State<TenantScreen> { // les données + l'affichage
  CanalEtat? _canal; // change quand une nouvelle réponse arrive
  ...
}
```

setState(() { ... })

C'est **LA** instruction centrale de Flutter : elle dit "j'ai changé une donnée, redessine l'écran maintenant". Sans elle, même si la donnée a changé en mémoire, l'écran resterait affiché tel quel.

```
setState(() {
  _canal = canal; // change la donnée...
});              // ...et Flutter redessine automatiquement l'écran
```

build(BuildContext context)

Chaque widget a une méthode build qui décrit “à quoi ressemble cet écran, en fonction des données actuelles”. Flutter l’appelle automatiquement à chaque setState. C’est le cœur de chaque fichier d’écran — un grand arbre de widgets imbriqués les uns dans les autres (Scaffold contient Padding, qui contient Column, qui contient des Text, etc.), comme des poupées russes.

Partie 2 — models.dart : la traduction du JSON en objets Dart

```
class CanalEtat {  
  final int canal;  
  final String role;  
  final double courantA;  
  ...  
}
```

Reproduit exactement les champs que le firmware renvoie dans /api/etat (voir document firmware, fonction construireJsonEtat()). Chaque objet CanalEtat = une fiche d’un canal.

```
factory CanalEtat.fromJson(Map<String, dynamic> json) {  
  return CanalEtat(  
    canal: json['canal'] as int,  
    courantA: (json['courant_A'] as num).toDouble(),  
    ...  
  );  
}
```

Un factory est une façon spéciale de construire un objet. fromJson prend une Map (l’équivalent Dart d’un dictionnaire — ce que devient un objet JSON une fois lu) et va chercher chaque champ par son nom (json['courant_A']) pour remplir un vrai objet CanalEtat typé. as int, as num : convertit/vérifie le type de la valeur lue.

EtatSysteme fait la même chose, mais pour l’objet englobant qui contient la tension secteur, le tarif, **et la liste** de tous les CanalEtat :

```
final liste = (json['canaux'] as List)  
  .map((c) => CanalEtat.fromJson(c as Map<String, dynamic>))  
  .toList();
```

.map(...) = “pour chaque élément de la liste, applique cette transformation” — ici, transforme chaque bout de JSON brut en un vrai objet CanalEtat. .toList() referme le résultat en une vraie liste Dart. C’est l’équivalent de .map() en JavaScript, ou d’une compréhension de liste [f(x) for x in liste] en Python.

Partie 3 — api_client.dart : parler au firmware

```
class ApiClient {  
  final String baseUrl;  
  ApiClient(this.baseUrl);
```

baseUrl = l’adresse de l’ESP32 (par exemple http://192.168.4.1), donnée à la création de l’objet. ApiClient(this.baseUrl) est un constructeur raccourci : this.baseUrl veut dire

“prends le paramètre reçu et mets-le directement dans le champ `baseUrl` de l’objet”.

```
Future<EtatSysteme> fetchEtat() async {
  final uri = Uri.parse('$baseUrl/api/etat');
  final reponse = await http.get(uri).timeout(const Duration(seconds: 5));
  if (reponse.statusCode != 200) {
    throw Exception('Erreur serveur : ${reponse.statusCode}');
  }
  final data = jsonDecode(reponse.body) as Map<String, dynamic>;
  return EtatSysteme.fromJson(data);
}
```

- `Uri.parse(...)` construit l’adresse complète, ex : `http://192.168.4.1/api/etat`
- `http.get(uri)` envoie une requête HTTP GET (comme quand ton navigateur charge une page) — exactement la même requête que fait `fetch()` dans `app.js` côté web
- `.timeout(...)` : abandonne et signale une erreur si l’ESP32 ne répond pas en 5 secondes (évite que l’appli reste bloquée indéfiniment)
- `reponse.statusCode != 200` : vérifie que le serveur a bien répondu “succès” (code 200) et pas une erreur
- `throw Exception(...)` : déclenche une erreur qui sera attrapée plus loin par un bloc `try/catch` (voir Partie 4)
- `jsonDecode(...)` transforme le texte JSON reçu en `Map`, puis `EtatSysteme.fromJson(...)` la transforme en objet Dart utilisable

```
Future<void> recharger(int canal, double montant) async {
  final uri = Uri.parse('$baseUrl/api/recharge?canal=$canal&montant=$montant');
  final reponse = await http.get(uri, headers: _headersAdmin).timeout(...);
  ...
}
```

Même principe, mais pour la recharge : construit l’URL avec les paramètres `canal` et `montant` directement dedans (comme dans un navigateur), et ajoute les en-têtes d’authentification :

```
Map<String, String> get _headersAdmin {
  final creds = base64Encode(utf8.encode('$adminUser:$adminPass'));
  return {'Authorization': 'Basic $creds'};
}
```

Ceci construit l’en-tête HTTP d’authentification “Basic Auth” attendu par le firmware (`request->authenticate(...)` côté ESP32, voir document firmware) : le couple utilisateur/mot de passe est encodé en base64 (un simple encodage de texte, pas un chiffrement) et envoyé dans l’en-tête `Authorization`.

Partie 4 — Les écrans

`home_screen.dart` — l’écran d’accueil

```
final TextEditingController _baseUrlController =
  TextEditingController(text: "http://192.168.4.1");
```

Un `TextEditingController` retient ce que l’utilisateur tape dans un champ de texte (ici, l’adresse IP du serveur) et permet de le relire.

```
Wrap(
  children: List.generate(nombreCanaux, (i) {
    return OutlinedButton(
```

```

        onPressed: () { Navigator.push(context, MaterialPageRoute(
          builder: (_) => TenantScreen(baseUrl: baseUrl, canalIndex: i),
        )); },
        child: Text(i == 0 ? "Canal 0 (bailleur)" : "Canal $i"),
      );
    }),
  )

```

List.generate(5, (i) => ...) crée 5 boutons d'un coup (un par canal), sans les écrire à la main un par un — équivalent d'une boucle for qui construirait une liste.

Navigator.push(context, MaterialPageRoute(builder: (_) => TenantScreen(...)))

c'est **la navigation** entre écrans en Flutter. Navigator.push empile un nouvel écran par-dessus l'actuel (comme ouvrir une nouvelle page — et il y a un bouton retour automatique). MaterialPageRoute(builder: ...) décrit quel écran construire (ici, TenantScreen, en lui passant l'IP du serveur et le numéro du canal choisi).

tenant_screen.dart — l'écran locataire

```

@override
void initState() {
  super.initState();
  _api = ApiClient(widget.baseUrl);
  _rafraichir();
  _timer = Timer.periodic(const Duration(seconds: 2), (_) => _rafraichir());
}

```

initState() s'exécute **une seule fois**, quand l'écran s'ouvre. Ici : crée le client API, va chercher les données une première fois (_rafraichir()), puis programme un rafraîchissement automatique toutes les 2 secondes avec Timer.periodic — l'équivalent Dart de setInterval() en JavaScript.

```

@override
void dispose() {
  _timer?.cancel();
  super.dispose();
}

```

dispose() s'exécute quand l'écran se **ferme**. _timer?.cancel() : ?. veut dire "si _timer n'est pas null, appelle .cancel() dessus" — arrête le rafraîchissement automatique pour ne pas continuer à faire des requêtes réseau inutiles une fois l'écran quitté (ça gaspillerait de la batterie et des ressources).

```

Future<void> _rafraichir() async {
  try {
    final etat = await _api.fetchEtat();
    final canal = etat.canaux.firstWhere((c) => c.canal == widget.canalIndex);
    if (!mounted) return;
    setState(() { _canal = canal; _erreur = null; });
  } catch (e) {
    if (!mounted) return;
    setState(() => _erreur = "Connexion impossible : $e");
  }
}

```

- `try { ... } catch (e) { ... }` : *“essaie ce bloc ; s’il y a une erreur, exécute plutôt le bloc catch au lieu de planter l’application”* — indispensable pour le réseau, qui peut échouer à tout moment (Wi-Fi hors de portée, ESP32 éteint...)
- `.firstWhere((c) => c.canal == widget.canalIndex)` : *“trouve, dans la liste de tous les canaux reçus, celui dont le numéro correspond à l’écran actuel”*
- `if (!mounted) return;` : vérifie que l’écran est toujours affiché avant de le mettre à jour (si l’utilisateur a déjà quitté l’écran pendant que la requête réseau était en cours, on ne fait rien — sinon Flutter afficherait une erreur)

Le `build()` de cet écran utilise un opérateur ternaire imbriqué (déjà vu en C++ : `condition ? siVrai : siFaux`) pour choisir entre trois affichages : message d’erreur, roue de chargement (`CircularProgressIndicator`), ou les vraies données.

admin_screen.dart — l’écran bailleur/admin

Même structure que `tenant_screen.dart` (`initState`, `Timer.periodic`, `_rafraichir`), avec en plus le formulaire de recharge :

```
Future<void> _envoyerRecharge() async {
  final montant = double.tryParse(_montantController.text);
  if (montant == null || montant <= 0) return;
  try {
    await _api.recharger(_canalSelectionne, montant);
    _montantController.clear();
    await _rafraichir();
    ...
  } catch (e) { ... }
}
```

`double.tryParse(texte)` essaie de convertir le texte tapé en nombre ; il rend `null` si ce n’est pas un nombre valide (au lieu de planter). On vérifie ensuite que le montant est positif avant d’envoyer la recharge. Après succès, `_montantController.clear()` vide le champ de texte, et `_rafraichir()` recharge immédiatement les données affichées (au lieu d’attendre le prochain passage automatique du `Timer`).

```
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text("Recharge enregistree")),
);
```

Affiche un petit message temporaire en bas de l’écran (une “snackbar”) — la confirmation visuelle que la recharge a bien été prise en compte.

```
...etat.canaux.map(_carteCanal),
```

`...` (spread operator) : *“insère chaque élément de cette liste directement ici, plutôt que la liste elle-même”* — nécessaire car `ListView` attend une liste de widgets, et `.map(_carteCanal)` produit justement une carte par canal (fonction définie plus bas dans le même fichier).

main.dart — le point de départ

```
void main() {
  runApp(const TfcApp());
}

class TfcApp extends StatelessWidget {
  @override
```

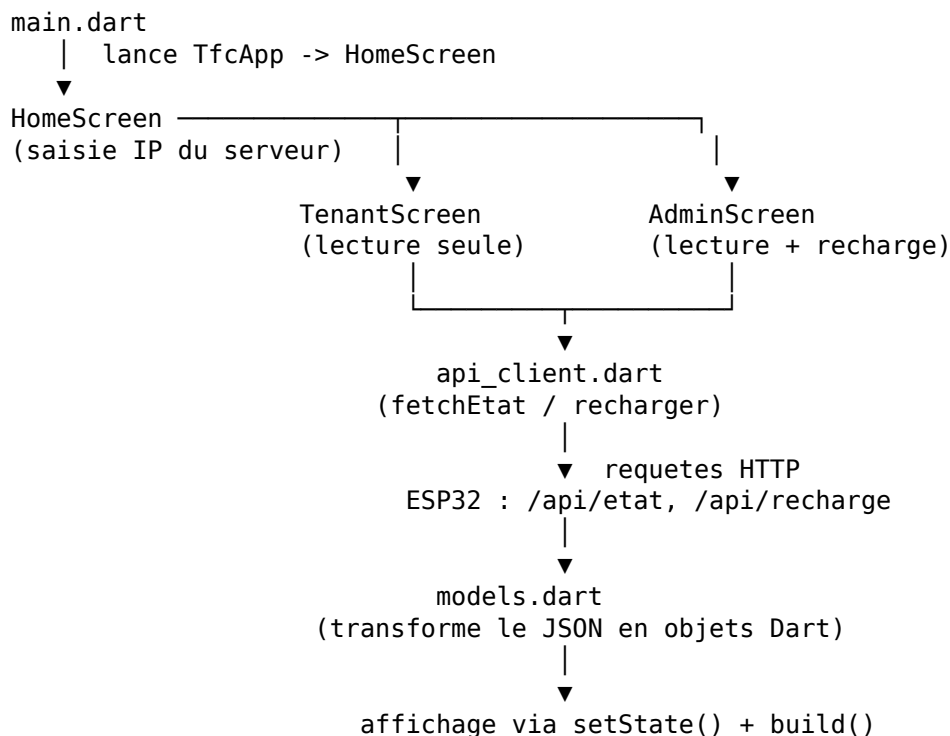
```

Widget build(BuildContext context) {
  return MaterialApp(
    title: 'TFC Multicanal',
    theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue)),
    home: const HomeScreen(),
  );
}

```

main() est la toute première fonction exécutée (comme setup() côté firmware, mais pour l'application). runApp(...) démarre Flutter avec le widget racine TfcApp. MaterialApp configure les réglages globaux (thème de couleurs, titre) et indique que le premier écran affiché (home) est HomeScreen.

Partie 5 — Comment tout s'articule



Chaque écran (Tenant/Admin) suit le même cycle : initState() lance un Timer qui rappelle _rafraichir() toutes les 2 secondes ; _rafraichir() appelle api_client.dart ; la réponse JSON est transformée en objets par models.dart ; setState() déclenche le redessin de l'écran avec les nouvelles valeurs.

Glossaire rapide

Mot	Signification simple
Widget	Tout élément affiché à l'écran (bouton, texte, écran entier)

Mot	Signification simple
StatelessWidget	Un widget qui ne change jamais après affichage
StatefulWidget	Un widget qui peut changer (et se redessiner)
setState(...)	"J'ai changé une donnée, redessine l'écran"
build(...)	La méthode qui décrit à quoi ressemble un widget
Future<T>	Une valeur de type T qui arrivera plus tard (opération asynchrone)
async / await	Marque et attend une opération qui prend du temps (réseau...)
Map<String, dynamic>	L'équivalent Dart d'un dictionnaire (issu d'un JSON)
factory	Une façon spéciale de construire un objet
Navigator.push(...)	Ouvre un nouvel écran par-dessus l'actuel
Timer.periodic(...)	Répète une action toutes les X secondes
try / catch	Essaie un bloc, récupère l'erreur si ça échoue au lieu de planter
? après un type	Cette variable peut être vide (null)
\$variable dans un texte	Insère la valeur directement dans le texte affiché